AI Forward Deployed Engineer — Six-Project LocalStack Program

> **Status: candidate proposal, not the program of record.**

Executive summary

This candidate program prepares experienced software engineers to work as AI Forward Deployed Engineers: discovering ambiguous client problems, scoping delivery, building and evaluating production-shaped AI systems, managing stakeholders, and handing a bounded solution into operation. It is project-first and includes all just-in-time theory, implementation, documentation, review, and defense work within **22 weeks × 20 hours/week = 440 hours**.

Projects 1–5 run locally with Docker Compose. LocalStack Hobby replaces selected infrastructure containers beginning in Project 2; students use individually owned accounts for personal, non-production educational work. Projects 1–5 do not require a real AWS account or expose a public endpoint. Project 6 first passes the same local acceptance path and then deploys one temporary, protected endpoint in a course-managed AWS sandbox. The total per-student program allowance remains **$20 for approved LLM APIs and $180 for AWS**.

This is not an entry-level or zero-to-hero program. Admission requires a practical assessment of Python, Git, REST APIs, SQL and relational databases, Docker, async service behavior, debugging an unfamiliar instrumented codebase, written stakeholder communication, and named cloud concepts. Customer-facing grading is asynchronous and deterministic, using role cards, fixed transcripts, checkpoint-triggered injects, evidence templates, recorded or written defenses, published acceptance floors, and common rubric anchors. An optional stakeholder chatbot may be used only for unassessed practice.

Required local environment

- **Hardware:** 16 GB RAM minimum using the assessed reduced profile; 32 GB recommended; 8 CPU cores and 80 GB free disk. Local GPU hardware is optional and never affects grading.
- **Operating system:** macOS, Linux, or Windows with WSL2.
- **Runtime profiles:** every project ships one pinned assessed Compose profile that must remain within a 10 GB container-memory budget. Optional observability or exploration profiles are started separately and are never required for grading.
- **Inference path:** a quantized model of no more than 3B parameters through Ollama is the assessed local path. The deterministic provider emulator is the grading fallback when local inference is unavailable. Local vLLM is optional exploration and cannot supply assessed latency or SLO evidence.
- **Admission preflight:** candidates run a supplied hardware, container, image-pull, and smoke-test check before enrollment. This is an admission requirement, not an uncounted program week.

Production-readiness dependencies

Before cohort authoring begins, the program must publish the scenario-pack contract, a versioned supplied-asset inventory, one `FIDELITY.md` per emulator or substitute service, six project rubrics with binary Must-Haves and Recommendations, pinned starter images, and the Project 6 verifier schema. These are program-build dependencies and do not add student hours.

LocalStack delivery stack

- **Orchestration and application runtime:** Docker Compose, LocalStack, Python, FastAPI, PostgreSQL, and Redis.
- **Object and document storage:** LocalStack S3 through the supplied `ObjectStore` adapter.
- **Search and retrieval:** PostgreSQL/pgvector and PostgreSQL full-text search with supplied configuration.
- **Messaging:** LocalStack SQS with one dead-letter queue through the supplied `JobQueue` adapter.
- **Identity and secrets:** preconfigured development-mode Keycloak and LocalStack SSM Parameter Store through the supplied `SecretProvider` adapter.
- **AI:** LangGraph, Ollama, manual Ragas evaluation, and Arize Phoenix.
- **Observability and security:** OpenTelemetry, Prometheus, Grafana, Tempo, Semgrep, Trivy, and Gitleaks.
- **Cloud transition:** Project 6 replaces local endpoint URLs and sentinel credentials with supplied real-AWS configuration and adapters; students do not build a general cloud platform.

LocalStack adoption and fidelity boundary

- **Conditional status:** promotion requires written confirmation that mandatory student use in this paid program is compatible with the assumed Hobby terms. Until then, the open-source variant is the launch-safe baseline.
- Students create and control individual accounts and tokens. A student who cannot obtain a token may use the open-source variant without penalty.
- LocalStack is introduced in Project 2. It is not used for Project 1 performance diagnosis.
- Projects 2–5 use only `awslocal`; plain AWS CLI is not used. Compose injects sentinel credentials and region values, the shared client factory injects the local endpoint, and a CI gate rejects SDK clients constructed outside that factory. Host AWS credential directories are never mounted.
- Deterministic initialization scripts recreate local state. No task assumes Hobby persistence.
- Local evidence does not prove AWS performance, durability, authorization enforcement, encryption guarantees, service quotas, or network behavior. Every assessed LocalStack service ships a `FIDELITY.md` naming the supported API subset and limitations.
- LocalStack substitutes for an open-source infrastructure container; it does not add an extra assessed deliverable or additional student hours.

Program and project summary

Project	Weeks	Hours	Engagement outcome
1. Client Discovery and System Diagnostics	1–3	60	Diagnose an unreliable AI-enabled workflow and recommend a bounded next decision.
2. Enterprise Data Integration and Hybrid RAG	4–7	80	Deliver and defend a retrieval system over messy enterprise data.
3. Reliable AI Service Delivery	8–11	80	Stabilize an AI service under provider failures and changing requirements.
4. Security, Guardrails and Governance Approval	12–15	80	Secure a high-value AI workflow and obtain a defensible risk decision.
5. Bounded Agent Workflow Under Ambiguity	16–19	80	Build and evaluate one bounded, auditable agent workflow.
6. AWS Capstone, Adoption and Handback	20–22	60	Deploy the accepted system temporarily, prove value and safety, hand it back, and tear it down.
Total	**22**	**440**	**Six sequential field-delivery engagements.**

Program delivery contract

- Every task estimate includes just-in-time theory, implementation, documentation, review, and defense.
- Every project produces a client-facing decision or communication artifact.
- Projects 1–5 are locally runnable and graded; no public endpoint or cloud deployment is allowed.
- The program follows one continuous product lineage across five physical starter repositories. Project 1 starts from repository 1. Before each of Projects 2–5, TripleTen provides the next repository from the correct reference solution to the previous project, so an earlier student defect does not propagate. Project 5 produces the capstone release tag in repository 5; Project 6 deploys, accepts, hands back, and tears down that exact tagged release.
- Published and held-out scenarios use the same interface but separate evidence. Reviewers grade decisions and evidence, not improvisational performance.
- Student-defined success criteria, SLOs, and acceptance conditions must meet or exceed a published project floor. Grades use the common floor plus the quality of the student's justification.
- Exactly one assessed implementation is pinned per capability. Named alternatives are optional exploration and never affect grading or reviewer support.
- Five graded recordings are required: the Project 2 SoW defense, Project 4 executive risk decision, Project 5 technical defense, and Project 6 technical and executive defenses. Each is a single take of at most 10 minutes; production polish is not assessed. Other defenses are written.
- Optional live discovery and constraints-change drills may run during office hours, but attendance and performance do not affect completion or grading.
- Application code uses conformance-tested `ObjectStore`, `JobQueue`, `SecretProvider`, and `ModelProvider` ports. Vendor SDK clients remain inside supplied adapters; Project 6 changes adapters and configuration rather than business logic.
- LocalStack teaches selected AWS API contracts and endpoint configuration through portable interfaces. Students must state what local evidence does not prove about the corresponding real AWS service.
- Projects 1–4 use no hosted LLM API. The program-issued, metered $20 allowance is split into $5 for a fixed Project 5 manual evaluation sample and $15 for Project 6 endpoint, held-out, and defense traffic. Students never use personal API accounts or payment methods.
- The program provisions no AWS GPU. Fine-tuning implementation is out of scope; students explain when it would be justified and why retrieval, prompting, or tool constraints are the correct levers for the assessed workflow.
- The five portfolio artifacts are: a traced client workflow and diagnosis, an undocumented legacy-integration specification and defensive client, unhappy-path resilience evidence, a deterministic evaluation suite with manual judged analysis, and a bounded agent with an audit trail.

Repository checkpoint model

Project	Physical repository	Starting checkpoint
1	Repository 1	Supplied initial legacy system
2	Repository 2	TripleTen reference solution through Project 1
3	Repository 3	TripleTen reference solution through Project 2
4	Repository 4	TripleTen reference solution through Project 3
5	Repository 5	TripleTen reference solution through Project 4
6	Repository 5	Student's accepted Project 5 capstone release tag

At each reset, students receive the correct prior reference solution plus their own accepted client-facing artifacts and decision records. They compare the reference checkpoint with their prior implementation, but previous code defects cannot block the next project.

Project 1 — Client Discovery and System Diagnostics

Project summary

Students enter an unreliable AI-enabled enterprise workflow as an FDE, not as a greenfield developer. They clarify the business problem, navigate a supplied local system, diagnose a seeded failure, and defend an evidence-backed recommendation. The emphasis is discovery, evidence triage, ownership, and communication under ambiguity.

- **Schedule:** Weeks 1–3 / 60 hours
- **Local environment:** the pinned reduced Docker Compose profile with FastAPI, PostgreSQL, Redis, OpenTelemetry, Prometheus, Grafana, and Tempo.
- **Project evidence:** discovery record, repaired system, before/after evidence, ADR, recommendation playback, and diagnostic handoff.

Task 1.1 — Engagement kickoff and stakeholder discovery (14 hours)

- **[Summary]** Establish what the client is trying to achieve before investigating implementation details.
- **[Skills]** Customer discovery, active listening, problem framing, stakeholder mapping, assumption management, success criteria.
- **[Tools]** Asynchronous role cards, fixed transcript, checkpoint-triggered injects, discovery template, decision log.
- Trace users, workflow, desired business outcome, constraints, and known exceptions.
- Separate reported symptoms, assumptions, and facts requiring technical validation.
- Capture open questions, decision owners, competing priorities, and measurable success criteria.
- Produce a discovery record from a deterministic asynchronous simulation.

Task 1.2 — Runtime orientation and evidence triage (10 hours)

- **[Summary]** Understand the supplied platform quickly enough to investigate one important request path.
- **[Skills]** Codebase orientation, request tracing, log and metric interpretation, architecture reading, technical prioritization.
- **[Tools]** Pinned Docker Compose profile, FastAPI, PostgreSQL, Redis, OpenTelemetry, Prometheus, Grafana, Tempo.
- Run the supplied API, worker, database, cache, and observability services locally.
- Follow one request through code, logs, metrics, and a distributed trace.
- Use the supplied architecture map rather than producing a complete C4 model.
- Treat monolith-versus-microservices as a decision prompt, not a baseline theory module.

Task 1.3 — Diagnose and repair the seeded failure (14 hours)

- **[Summary]** Reproduce the client-visible failure and make one evidence-backed repair.
- **[Skills]** Reproduction, hypothesis testing, bottleneck analysis, observability repair, validation, causal reasoning.
- **[Tools]** Pytest, k6, OpenTelemetry, Prometheus, Grafana, Tempo, deterministic provider emulator.
- Run one bounded, deterministic load and provider-latency scenario.
- Repair one trace, metric, or dashboard defect selected by the scenario.
- Identify the first supported bottleneck and reject one plausible false explanation.
- Re-run the same scenario and record before-and-after evidence.

Task 1.4 — Problem framing and recommendation playback (12 hours)

- **[Summary]** Translate technical evidence into a decision the stakeholder can act on.
- **[Skills]** Technical synthesis, option framing, effort estimation, business communication, audience adaptation.
- **[Tools]** Recommendation template, lightweight estimation worksheet, architecture decision record.
- Rewrite the initial problem statement using verified workflow evidence.
- Present bounded alternatives, including a no-AI or process-change option when appropriate.
- Estimate effort and capacity using explicit assumptions.
- Record separate engineering-detail and business-impact explanations.

Task 1.5 — Diagnostic handoff (10 hours)

- **[Summary]** Leave the client with an auditable diagnosis and a bounded next decision.
- **[Skills]** Ownership, handoff writing, risk communication, recommendation defense, scope control.
- **[Tools]** Git, pull request, ADR, evidence packet, timed written defense.
- Submit the repair, evidence packet, concise ADR, and updated workflow trace.
- Record unresolved risks, excluded work, and next discovery questions.
- Respond in writing to a timed stakeholder challenge without inventing evidence.
- Defend ownership of the recommendation through an evidence-linked decision record.

Project 2 — Enterprise Data Integration and Hybrid RAG

Project summary

Students scope and deliver a retrieval system over messy client documents without turning the engagement into a platform rewrite. They work through data ownership and access constraints, construct a reproducible ingestion path, implement hybrid retrieval, evaluate failure modes, and defend a statement of work.

- **Schedule:** Weeks 4–7 / 80 hours
- **Local environment:** Docker Compose, LocalStack S3, PostgreSQL/pgvector, PostgreSQL full-text search, supplied document parser, local embeddings, and a supplied cross-encoder.
- **Project evidence:** data discovery record, ingestion report, retrieval API, golden set, evaluation report, and defended SoW.

Task 2.1 — Data and use-case discovery (12 hours)

- **[Summary]** Determine whose decision retrieval supports and which data can be used safely.
- **[Skills]** Data discovery, domain interviewing, ownership mapping, access analysis, acceptance-criteria design.
- **[Tools]** Stakeholder role cards, data inventory, access matrix, golden-query template.
- Analyze asynchronous responses from domain, data-owner, and compliance stakeholders.
- Document source ownership, access constraints, freshness needs, and failure consequences.
- Define measurable retrieval acceptance criteria before choosing technical methods.
- Identify one process or data-quality problem that RAG should not conceal.

Task 2.2 — Messy-document ingestion (20 hours)

- **[Summary]** Build a reproducible ingestion pipeline for a deliberately bounded enterprise document set.
- **[Skills]** Ingestion design, S3 API integration, endpoint-safe SDK configuration, metadata normalization, deduplication, chunking, embedding, data-quality validation.
- **[Tools]** LocalStack S3, `awslocal`, supplied client factory, supplied PDF/HTML parser, PostgreSQL, Python, local embedding model, Docker Compose.
- Use the supplied robust parser with text-based PDFs and clean, bounded table structures; OCR library selection is out of scope.
- Store source documents in LocalStack S3 through the supplied `ObjectStore` adapter and normalize metadata, duplicate identifiers, malformed content, and access labels.
- Implement bounded chunking and local embedding through the supplied pipeline scaffold.
- Produce ingestion-quality evidence for published and held-out fixtures.
- Cite the `ObjectStore` fidelity note and state which real-S3 behaviors the LocalStack result does not prove.

Task 2.3 — Hybrid retrieval service (24 hours)

- **[Summary]** Implement the smallest production-shaped retrieval boundary that meets the agreed use case.
- **[Skills]** Dense and sparse retrieval, reranking, authorization metadata propagation, API extension, migration use.
- **[Tools]** PostgreSQL/pgvector, PostgreSQL full-text search, supplied reciprocal-rank fusion and cross-encoder wrappers, FastAPI, Alembic.
- Combine pgvector dense retrieval with PostgreSQL full-text search through the supplied fusion function.
- Rerank hybrid candidates locally before returning the final result set.
- Preserve source metadata and authorization attributes through retrieval.
- Extend the supplied versioned API and reuse provided migrations, repositories, and connection pooling.
- Choose and justify candidate depth, fusion parameters, and the rerank cut-off; show their measured latency and quality effects.

Task 2.4 — Retrieval evaluation and failure analysis (14 hours)

- **[Summary]** Demonstrate when retrieval succeeds, when it fails, and why.
- **[Skills]** Golden-set design, retrieval metrics, latency analysis, error attribution, evidence-based iteration.
- **[Tools]** Pytest, evaluation fixtures, recall/precision metrics, OpenTelemetry, supplied reporting script.
- Build a compact golden query set with relevance expectations.
- Measure retrieval quality and latency against agreed criteria.
- Attribute misses to parsing, chunking, metadata, dense retrieval, keyword retrieval, or reranking.
- Recommend one bounded improvement supported by evidence.
- Meet or exceed the published retrieval-quality and latency floor on held-out fixtures.

Task 2.5 — Scope and statement-of-work defense (10 hours)

- **[Summary]** Convert discovery and technical findings into a delivery agreement.
- **[Skills]** SoW writing, scope control, estimation, negotiation, change rejection, acceptance definition.
- **[Tools]** SoW template, estimation worksheet, assumption log, timed stakeholder inject, recorded defense.
- Define outcomes, deliverables, assumptions, exclusions, acceptance criteria, and responsibilities.
- Estimate the next increment and identify its largest uncertainty.
- Process a stakeholder request that expands scope without moving time or budget.
- Record the graded SoW defense, including what will not be built and the resulting decision.

Project 3 — Reliable AI Service Delivery

Project summary

Students stabilize a retrieval-enabled AI service whose provider behavior and client priorities change under load. They operate local inference and deterministic provider paths, implement a bounded resilience pattern, connect it to an SLO, renegotiate a changed requirement, and hand over an executable incident process.

- **Schedule:** Weeks 8–11 / 80 hours
- **Local environment:** Docker Compose, Ollama, deterministic provider and undocumented legacy-service emulators, LocalStack SQS/DLQ, OpenTelemetry, Prometheus, and Grafana.
- **Project evidence:** resilient service, scenario results, SLO and alert, failure timeline, approved change record, incident communications, and runbook.

Task 3.1 — Local AI runtime and service boundary (18 hours)

- **[Summary]** Operate the platform through portable provider boundaries and derive a defensive contract for one undocumented legacy service.
- **[Skills]** Provider abstraction, legacy integration, schema tolerance, permission and rate-limit discovery, fallback reasoning.
- **[Tools]** Docker Compose, Ollama, FastAPI, deterministic provider emulator, supplied undocumented legacy-service emulator.
- Run the supplied multi-service environment locally.
- Use one provider adapter across local inference and deterministic scenarios.
- Discover an inconsistent response schema, hidden rate limit, stale field, and denied operations through observed behavior.
- Produce a derived interface specification and a defensive client with schema tolerance and bounded backoff.
- Explain Kubernetes, managed inference, and fine-tuning only as architecture alternatives relevant to the service.

Task 3.2 — Provider resilience (20 hours)

- **[Summary]** Keep the client workflow safe during latency, rate-limit, malformed-response, and outage scenarios.
- **[Skills]** Deadlines, retry and backoff, circuit breaking, fallback, idempotency, SQS visibility-timeout and dead-letter handling.
- **[Tools]** LocalStack SQS/DLQ, `awslocal`, supplied client factory, provider emulator, supplied resilience library, Pytest, OpenTelemetry.
- Configure the supplied circuit-breaker-with-fallback pattern and use bounded retry only for scenario-approved transient failures.
- Enforce deadlines, structured failures, and idempotent behavior where required.
- Implement one LocalStack SQS dead-letter path through the supplied `JobQueue` adapter and submit evidence of failed-message recovery.
- Prove behavior through deterministic provider scenarios.
- Cite the `JobQueue` fidelity note and state which real-SQS delivery behaviors the LocalStack result does not prove.

Task 3.3 — SLO and failure lab (18 hours)

- **[Summary]** Connect one operational objective to a client-visible failure and recovery workflow.
- **[Skills]** SLO design, alerting, reversible failure testing, incident timelines, residual-risk analysis.
- **[Tools]** Prometheus, Grafana, OpenTelemetry, supplied failure injector, k6.
- Define one user-impact SLO and one actionable alert.
- Run one reversible failure lab and capture detection, mitigation, and recovery evidence.
- Validate the selected resilience behavior under failure.
- Document remaining failure modes without constructing a broad chaos suite.
- Meet or exceed the published availability and recovery floor; justify any stricter student-defined target.

Task 3.4 — Changing-requirements simulation (12 hours)

- **[Summary]** Re-plan delivery after a stakeholder changes a reliability requirement.
- **[Skills]** Clarification, impact analysis, negotiation, prioritization, change control, decision documentation.
- **[Tools]** Timed asynchronous inject, impact template, change log, timed written response.
- Clarify the outcome, urgency, decision owner, and hidden trade-off.
- Assess effects on scope, risk, schedule, and operational cost.
- Negotiate which commitment moves out when the new requirement moves in.
- Record the approved change rather than silently absorbing it.
- During the same task, triage one Project 2 client escalation against the active Project 3 commitment and record the context-switching trade-off.

Task 3.5 — Incident communication and operational handoff (12 hours)

- **[Summary]** Communicate a failure honestly and leave an executable recovery process.
- **[Skills]** Incident communication, audience adaptation, uncertainty communication, runbook design, escalation planning.
- **[Tools]** Incident templates, runbook template, timeline evidence, timed written briefing.
- Write separate engineering and stakeholder incident updates.
- Write a short stakeholder briefing while recovery information remains uncertain.
- Deliver a focused detection, mitigation, rollback, and escalation runbook.
- Convert the failure lesson into one reusable delivery pattern.

Project 4 — Security, Guardrails and Governance Approval

Project summary

Students secure one high-value AI workflow and make the result inspectable by engineering, security, compliance, and business stakeholders. They select proportionate controls, implement one bounded security boundary, test it against held-out attacks, and obtain a documented risk decision without overstating educational evidence.

- **Schedule:** Weeks 12–15 / 80 hours
- **Local environment:** preconfigured development-mode Keycloak, LocalStack SSM Parameter Store, Semgrep, Trivy, Gitleaks, OpenTelemetry, and supplied attack scenarios.
- **Project evidence:** stakeholder risk record, threat model, tested controls, control matrix, residual-risk register, and executive decision.
- **Evidence boundary:** the project demonstrates security-control design and integration against development services. It does not demonstrate production hardening, key custody, an audit, certification, or legal compliance. Every control-matrix row states what its evidence does and does not prove.

Task 4.1 — Security and governance discovery (12 hours)

- **[Summary]** Identify actors, sensitive data, decision rights, and risk tolerance.
- **[Skills]** Security discovery, workshop facilitation, conflict reconciliation, risk treatment, governance ownership.
- **[Tools]** Stakeholder role cards, data-flow template, risk register, asynchronous workshop injects.
- Reconcile engineering, security, compliance, and business expectations.
- Map sensitive data, availability needs, audit expectations, and decision rights.
- Distinguish risks requiring control, acceptance, transfer, or feature removal.
- Record conflicts and accountable decision owners.

Task 4.2 — Threat model and control selection (18 hours)

- **[Summary]** Model one workflow deeply enough to choose proportionate controls.
- **[Skills]** STRIDE, trust-boundary analysis, abuse-case design, prompt-injection analysis, control selection.
- **[Tools]** Threat-model template, data-flow diagram, OWASP LLM guidance, control decision record.
- Identify assets, actors, trust boundaries, and high-impact abuse cases.
- Emphasize prompt injection and unsafe output handling.
- Select controls using evidence and reject unnecessary complexity.
- Treat legal frameworks as engineering-control mapping, not certification.

Task 4.3 — Implement the security boundary (30 hours)

- **[Summary]** Implement and verify the smallest control set needed for the workflow.
- **[Skills]** OIDC/RBAC integration, prompt and output controls, PII redaction, audit logging, SSM parameter integration, CI security.
- **[Tools]** Preconfigured Keycloak realm, LocalStack SSM Parameter Store, `awslocal`, supplied client factory, FastAPI, supplied redactor, supplied pinned CI security workflow, Semgrep, Trivy, Gitleaks, Pytest.
- Protect one endpoint using the supplied development OIDC/RBAC scaffold.
- Add prompt-injection handling, output validation, PII redaction, and audit logging at one AI boundary.
- Move one hardcoded secret to LocalStack SSM Parameter Store through the supplied `SecretProvider` adapter and configure the supplied deterministic CI security gate.
- Test published and held-out attack and data-leakage scenarios; explain remaining gaps.
- State what the emulated parameter-store evidence does and does not prove about AWS secret custody and access enforcement.

Task 4.4 — Governance evidence and adversarial review (12 hours)

- **[Summary]** Make security claims inspectable by people who did not build the system.
- **[Skills]** Evidence mapping, adversarial review, AI-output verification, residual-risk writing, human-oversight design.
- **[Tools]** Control matrix, test evidence, risk register, supplied flawed AI-generated artifact.
- Link risks, controls, tests, evidence, and owners.
- Review a flawed AI-generated threat or compliance artifact for overclaims and omissions.
- Record residual risks and limits of the evidence.
- Define required human oversight and review triggers.

Task 4.5 — Executive risk decision (8 hours)

- **[Summary]** Obtain a go, conditional-go, or no-go decision without hiding uncertainty.
- **[Skills]** Executive communication, risk translation, boundary defense, decision facilitation, accountability.
- **[Tools]** Executive brief, recorded defense, timed pressure inject, decision record.
- Present value, material risks, implemented controls, and remaining exposure.
- Respond to pressure to describe educational evidence as production certification.
- Record the graded executive risk defense, recommend a decision, and defend its conditions.
- Record the outcome, owner, conditions, and review date.

Project 5 — Bounded Agent Workflow Under Ambiguity

Project summary

Students turn a vague automation request into one bounded, auditable LangGraph workflow. They define the authority boundary, implement the smallest useful agent flow, evaluate it with deterministic scenarios, process a late change, and defend the solution to technical and business audiences. This project begins from repository 5, which contains TripleTen's correct reference checkpoint through Project 4, and produces the release tag deployed in Project 6.

- **Schedule:** Weeks 16–19 / 80 hours
- **Local environment:** LangGraph, Project 2 retrieval, carried-forward LocalStack S3/SQS/SSM adapters, Ollama, Ragas, Arize Phoenix, PostgreSQL, and Docker Compose.
- **Project evidence:** workflow map, value hypothesis, bounded agent, audit trail, evaluation report, approved change, recorded technical defense, and written executive brief.

Task 5.1 — Ambiguous client brief and workflow discovery (12 hours)

- **[Summary]** Turn a vague automation request into an observable workflow and decision problem.
- **[Skills]** Workflow discovery, ambiguity management, exception analysis, authority design, assumption tracking.
- **[Tools]** Process map, stakeholder role cards, timed injects, assumption log, decision inventory.
- Trace current handoffs, exceptions, failure costs, and human decision points.
- Identify where deterministic software is safer or cheaper than an agent.
- Define authority, escalation conditions, prohibited actions, and decision owners.
- Update assumptions as stakeholder information changes.

Task 5.2 — Scope and value hypothesis (10 hours)

- **[Summary]** Select a narrow agent use case with an evidence-based business outcome.
- **[Skills]** Value framing, scope reduction, cost estimation, acceptance design, rollback planning, autonomy negotiation.
- **[Tools]** Value hypothesis canvas, cost model, SoW addendum, acceptance checklist.
- Define value through revenue, risk, time, or operating cost.
- Estimate delivery and operating cost from named assumptions.
- Set acceptance, exclusion, rollback, and human-override conditions.
- Defend a smaller scope when broad autonomy is requested.
- Meet or exceed the published authority, override, and rollback floor.

Task 5.3 — Bounded agent implementation (24 hours)

- **[Summary]** Implement one auditable workflow that cannot exceed its agreed authority.
- **[Skills]** State-graph design, tool integration, schema validation, sandboxing, budget controls, human-in-the-loop design.
- **[Tools]** LangGraph, Pydantic, Project 2 retrieval API, Ollama, PostgreSQL, Docker Compose.
- Build one bounded multi-step tool flow rather than a general agent platform.
- Reuse Project 2 retrieval and add step, token, request, and tool limits.
- Preserve tool calls, state transitions, failures, and human decisions in an audit trail.
- Fine-tuning implementation is out of scope. In the technical defense, explain when fine-tuning would be justified and why prompting, retrieval, or tool constraints are the appropriate levers for this workflow.

Task 5.4 — Agent evaluation and observability (18 hours)

- **[Summary]** Prove useful behavior and expose important failures without relying on demo impressions.
- **[Skills]** Scenario-set design, agent evaluation, trace analysis, cost analysis, judge calibration, failure interpretation.
- **[Tools]** Supplied deterministic metric script, Ragas, Arize Phoenix, OpenTelemetry, Pytest, cached fixtures, bounded LLM-as-judge worksheet.
- Cover success, refusal, escalation, and recovery in a compact golden scenario set.
- Run bit-reproducible CI checks for schema validity, tool-sequence conformance, recall, refusal, escalation, and recovery using cached fixtures.
- Use Ragas and an LLM-as-judge manually on one fixed 20-case sample; report variance across three runs. Neither gates CI.
- Use Phoenix for trace inspection, latency, cost, tool behavior, and user-visible failure.

Task 5.5 — Change control and dual-room defense (16 hours)

- **[Summary]** Revise the solution under pressure and defend it to technical and business audiences.
- **[Skills]** Change control, impact analysis, technical defense, executive storytelling, adoption-risk communication.
- **[Tools]** Change request, architecture evidence, evaluation report, recorded technical defense, written executive brief.
- Process a late stakeholder request through explicit analysis and approval.
- Explain architecture, safety boundaries, evaluation gaps, and rejected alternatives to engineers.
- Record the graded technical defense and submit a written executive brief covering value, limitations, adoption risk, and the next investment decision.
- Record approved change, deferred work, and uncertainty.

Project 6 — AWS Capstone, Adoption and Handback

Project summary

Students take the locally accepted Project 5 repository through a supplied, protected AWS deployment path. They agree on responsibilities and cost controls, demonstrate acceptance evidence, prepare users and operators, defend the engagement asynchronously, and prove teardown. The project teaches bounded cloud delivery—not general AWS administration.

- **Schedule:** Weeks 20–22 / 60 hours
- **Fixed cloud profile:** one course-managed AWS Organizations member account and one `t3.large` CPU host run Caddy, the application, worker, PostgreSQL/pgvector with full-text search, and Redis. S3 supplies object storage, SQS/DLQ supplies asynchronous delivery, and SSM Parameter Store supplies configuration. Keycloak, Phoenix, Prometheus, Grafana, LocalStack, MinIO, and RabbitMQ are not deployed.
- **Prohibited resources:** no GPU, NAT gateway, load balancer, managed database, managed search domain, second public IPv4 address, second always-on instance, student-created IAM principal, or untagged resource.
- **Cloud boundary:** one temporary protected endpoint and 14 consecutive days of reviewer access beginning at the first verified endpoint, which must occur by the end of Week 20 so review, one remediation, and teardown remain inside the 22-week program. Reviewer turnaround is three business days.
- **Budget:** $180 maximum for AWS. The normal target is at most $80; remaining headroom covers exactly one instructor-approved remediation deployment and operational variance. Current prices and the profile must be validated in a course sandbox before launch.
- **Access and cleanup:** the program supplies constrained launch templates, a constrained student role, budget actions, the DNS hostname, Caddy TLS configuration, Session Manager access, automated teardown, and the verifier. Students never use personal AWS accounts or payment methods.
- **Project evidence:** deployment agreement, protected endpoint, verifier report, acceptance decision, handback pack, recorded defenses, final cost, and teardown proof.

Task 6.1 — Capstone kickoff and deployment agreement (8 hours)

- **[Summary]** Agree on responsibilities, acceptance evidence, cost limits, and exit conditions before using AWS.
- **[Skills]** Deployment planning, RACI definition, readiness assessment, cost governance, acceptance design, escalation planning.
- **[Tools]** Readiness checklist, RACI, supplied budget report, delivery plan, review SLA, teardown contract.
- Review security, support, data, access, dependencies, and decision owners.
- Confirm the $15 Project 6 API allocation, $180 AWS cap, 14-day reviewer window, three-business-day review SLA, approved region, tags, and teardown deadline.
- Define milestones, responsibilities, escalation paths, acceptance criteria, and rollback/no-go threshold.
- Pass the local gate before provisioning cloud resources.
- Confirm that any failed Project 4 Must-Have has been remediated and re-verified before deployment.

Task 6.2 — Protected AWS deployment (18 hours)

- **[Summary]** Deploy the tagged, locally accepted release through the fixed course-managed AWS profile.
- **[Skills]** Local-to-cloud translation, migration-gap analysis, protected deployment, secrets handling, least privilege, cost-aware operation, release verification.
- **[Tools]** Supplied launch templates and deploy script, AWS CLI, ECR, EC2, S3, SQS/DLQ, SSM Parameter Store, CloudWatch, Caddy.
- Run the supplied deploy script and launch templates; students do not author general-purpose infrastructure.
- Produce the first verified endpoint within the first 20 Project 6 hours; remaining deployment-task time is reserved for control verification and stabilization.
- Replace local adapters with supplied AWS adapters without changing business logic.
- Produce a migration-delta record naming each local/AWS mapping, configuration change, AWS-only policy or network requirement, and one failure mode that local evidence could not prove.
- Configure the program-issued hostname and trusted TLS, reviewer token, 30 requests/minute/token, 128 KiB request bodies, HTTPS-only ingress, no public SSH, IMDSv2, controlled egress, tags, synthetic data, and bounded retention.
- Demonstrate and handle at least one real-AWS policy-denial, throttling, consistency, or network-reachability failure.

Task 6.3 — Acceptance, operations and cost evidence (12 hours)

- **[Summary]** Demonstrate that the deployed system meets technical and business acceptance conditions.
- **[Skills]** Acceptance testing, cloud observability, failure triage, cost analysis, remediation prioritization, stakeholder communication.
- **[Tools]** Course verifier, CloudWatch, supplied budget report, held-out scenario pack.
- Run held-out functional, security, resilience, and teardown-readiness checks.
- Show quality, latency, failure, and cost evidence against agreed criteria.
- Triage one simulated acceptance failure and communicate its delivery impact.
- Keep remediation inside the endpoint, assessment window, and budget.
- Apply the published evidence thresholds to derive accept, conditional-accept, or reject; one instructor-approved remediation deployment is permitted before automated teardown.

Task 6.4 — Adoption and operational handback (8 hours)

- **[Summary]** Prepare the client organization to operate, govern, and appropriately use the system.
- **[Skills]** Adoption planning, enablement, runbook design, ownership transfer, limitation communication, success measurement.
- **[Tools]** Operator runbook, user guide, escalation map, limitations register, timed written enablement scenario.
- Identify user groups, workflow changes, adoption risks, and responsible owners.
- Produce operator guidance, user guidance, escalation paths, and known limitations.
- Respond to skeptical-user questions through a deterministic timed written simulation.
- Define post-launch measures for usage, value, quality, and safety.

Task 6.5 — Technical and business defenses (6 hours)

- **[Summary]** Defend the completed engagement in two rooms using audience-appropriate evidence.
- **[Skills]** Technical defense, executive communication, evidence discipline, constraint response, acceptance negotiation.
- **[Tools]** Recorded defense, timed asynchronous scenario inject, architecture evidence, value brief, acceptance record.
- Record a technical defense covering architecture, controls, evaluations, operations, cost, and residual risks.
- Record an executive defense covering value, adoption status, limitations, and next investment.
- Respond via a recorded defense to a timed asynchronous scenario inject without inventing evidence or promising unapproved scope.
- Obtain a simulated accept, conditional-accept, or reject decision with rationale.

Task 6.6 — Teardown, portfolio and reusable field playbook (8 hours)

- **[Summary]** Close the engagement safely and turn customer-specific learning into reusable delivery knowledge.
- **[Skills]** Teardown verification, engagement closure, evidence retention, knowledge reuse, sensitive-data separation.
- **[Tools]** AWS CLI, supplied teardown script, cost report, resource inventory, field-playbook template.
- Remove and verify instances, volumes, public IPv4 addresses, tokens, parameters, traces, images, buckets, and endpoint records.
- Deliver final cost, teardown, acceptance, and unresolved-risk evidence.
- Separate reusable patterns from client-specific and sensitive information.
- Publish a sanitized capstone repository, reference-architecture write-up, discovery-to-handback playbook, and product-feedback record using synthetic data and excluding role-card content.
- Add the five auditable program artifacts to a public portfolio index; demo-day participation is optional and ungraded.

Workload verification

Project	Calculation	Hours
Project 1	3 weeks × 20	60
Project 2	4 weeks × 20	80
Project 3	4 weeks × 20	80
Project 4	4 weeks × 20	80
Project 5	4 weeks × 20	80
Project 6	3 weeks × 20	60
Program	**22 weeks × 20**	**440**